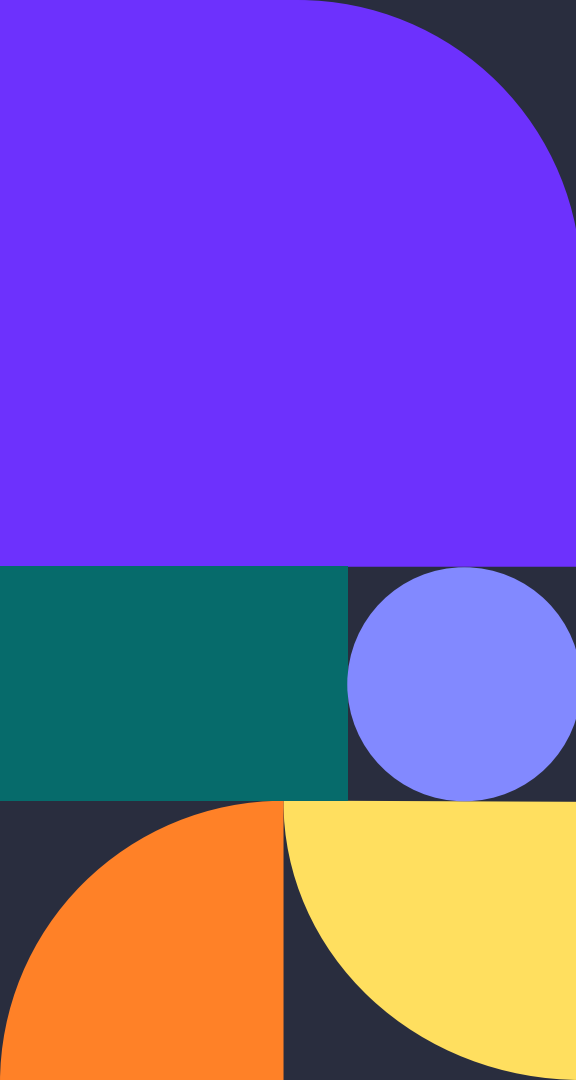
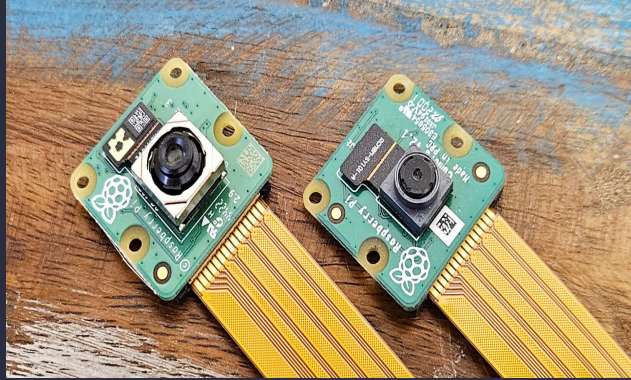




LAN Video & Audio Transceiver

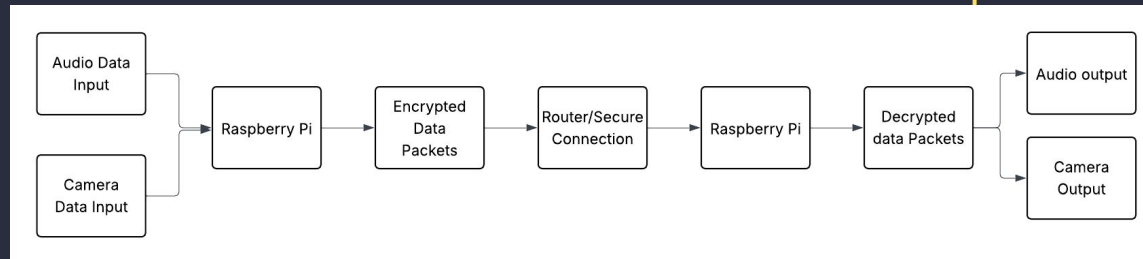
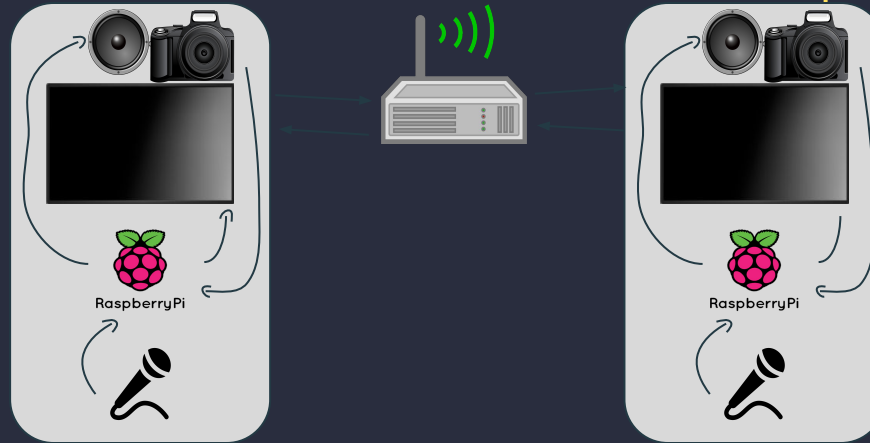
Sebastian Graciano, Erik Piche, Chris Valdivia

Overview



This project combines both receiving and transmitting capabilities to send a video stream and audio between two Raspberry pi 5 devices. Both the Audio and Video data are encrypted in packets using AES-256-CBC and OpenSSL to take full advantage of the Broadcom BCM2712 chip's cryptographic processing capabilities.

Project Phase: Configuration & Setup



Project phase: Material Preparation

Gathered all the necessary equipment for the project to commence.
Began the testing of the code live with the listed materials.

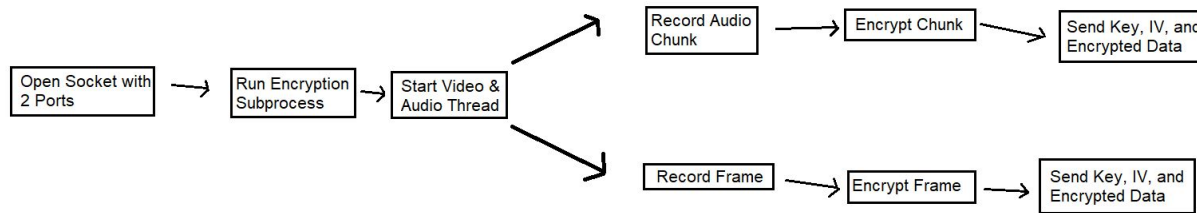
Materials:

- 2 Raspberry Pi 5s
- 2 Raspberry Pi Cams
- 2 USB microphones
- 2 wired Earphones/Headphones
- 2 USB to aux adapters
- 2 Raspberry Pi Displays

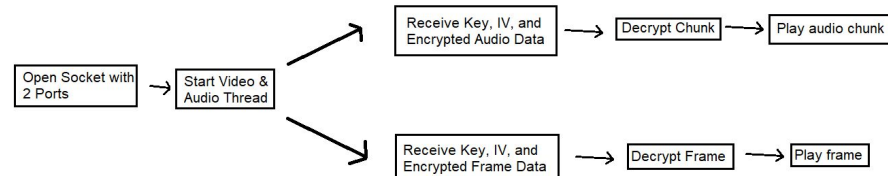


Code Abstraction:

Sender.py



Receiver.py



Why Use Rust For Encryption?

- Rust **eliminates common memory vulnerabilities** like buffer overflows and null pointer dereferencing at compile time, without needing a garbage collector. (simply manages memory better)
- Memory Bugs such as a memory leak can expose sensitive data
- Rust is very efficient compared to C
- Rust is easier for the programmer (No need to worry about memory leaks, memory allocation, and releasing memory.)

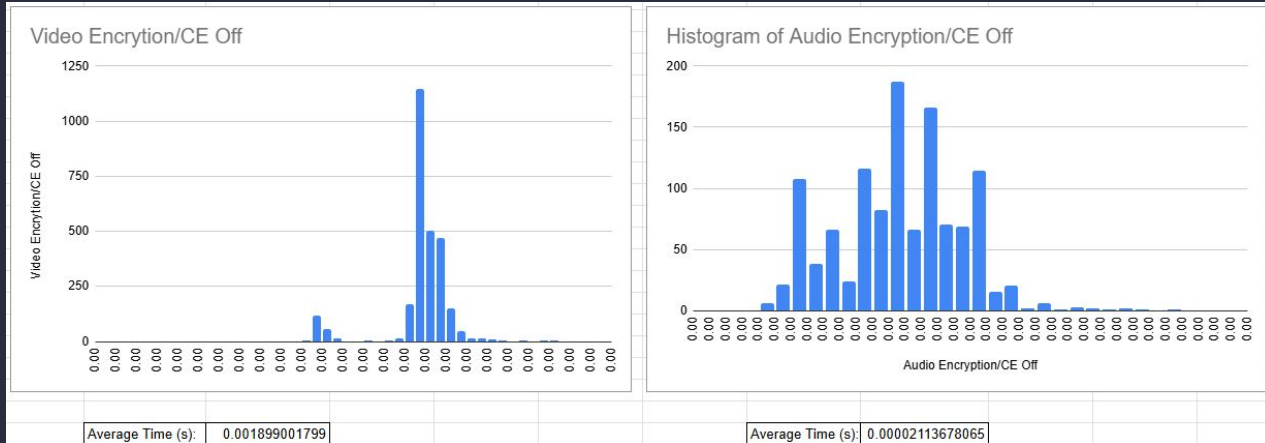
Encryption Type

AES 256bit CBC

- 256bit key
- 128bit IV (Initialization Vector)
- Requires Padding
- Cipher Block Chaining: **Encrypts** each block of plaintext **using the previous ciphertext block**
- Good for streams since we are sending each packet as "chunks" and that chunk can be used to encrypt another chunk

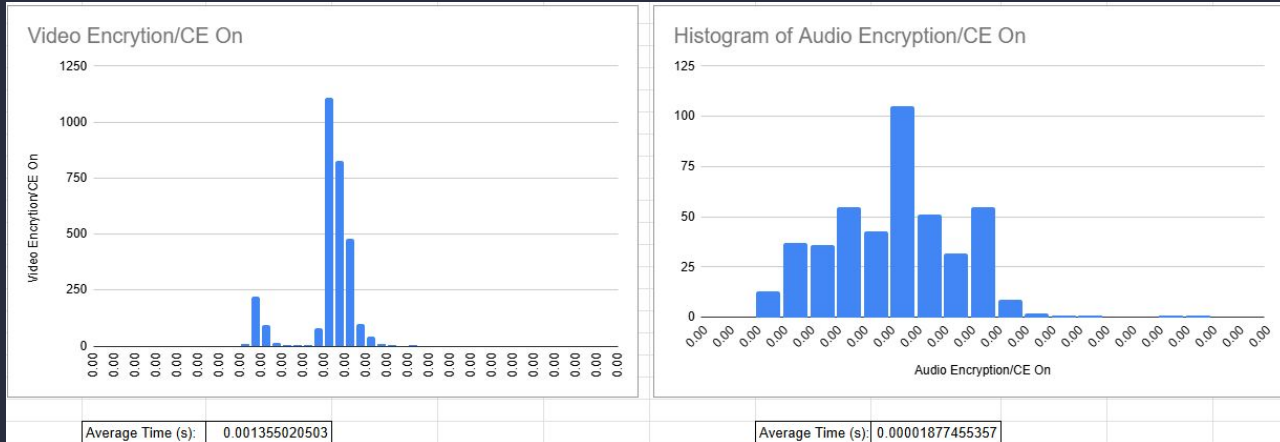
Encryption Time Stats: Crypto Engine (OFF)

Sender.py



Encryption Time Stats: Crypto Engine (ON)

Sender.py



Perf Stats:

Crypto Engine (OFF)

Performance counter stats for 'python3 Sender.py':

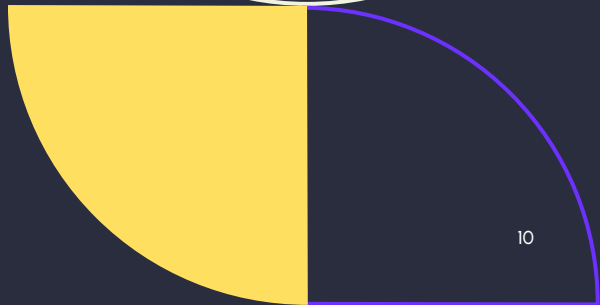
4,907,929,365	cycles	
5,403,443,368	instructions	# 1.10 insn per cycle

10.064802820 seconds time elapsed

1.930378000 seconds user

0.177320000 seconds sys

CPI: 0.91



Perf Stats:

Crypto Engine (ON)

Performance counter stats for 'python3 Sender.py':

4,767,204,604 cycles
5,760,117,513 instructions # 1.21 insn per cycle

10.068567642 seconds time elapsed

1.914582000 seconds user
0.197089000 seconds sys

CPI: 0.83

Perf Analysis:

	Without CE	With CE	Change
Cycles	4,907,929,365	4,767,204,604	✓ 140M fewer
Instructions	5,403,443,368	5,760,117,513	✗ 356M more
IPC (Insn per cycle)	1.10	1.21	✓ 0.11 more IPC
CPI	0.91	0.83	✓ 0.08 CPI Less

Why CPI < 1?

- Multiple threads can fill idle execution units better.
- Multiprocessing allows for the script to run on multiple cores
- The CPU can overlap execution of instructions from different threads.

Why More Instruction with CE?

- CE driver interactions
- Pipelining data into Rust

Power Stats

CE OFF	CE ON
MAX = 4.07 (W)	MAX = 3.89 (W)
MIN = 3.76 (W)	MIN = 3.73 (W)

Note:

- Based on a 20s runtime
- No averages were recorded

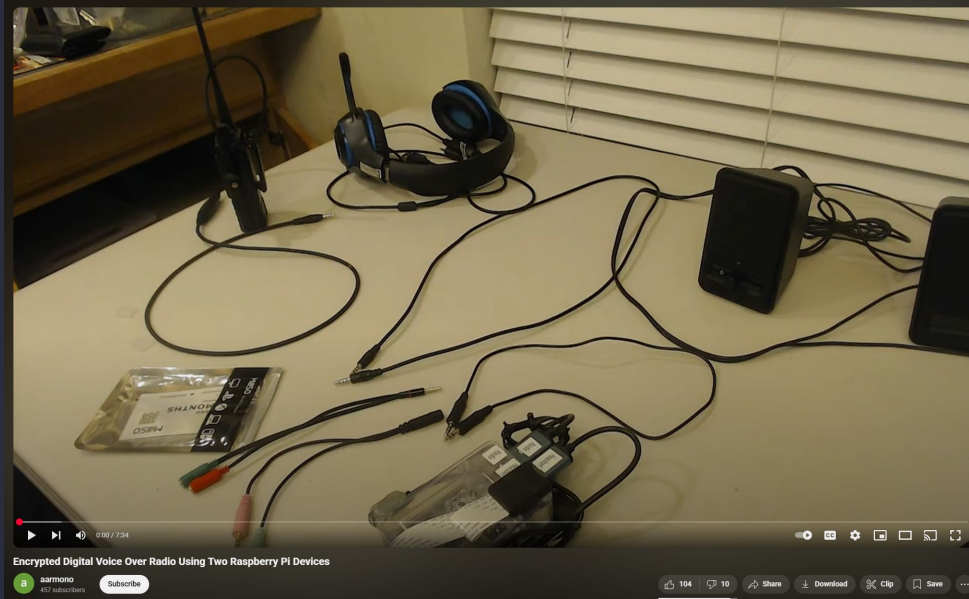
EDUROAM Latency

```
sebgra518@raspberrypi:~/ECE4301Final $ ping 10.110.156.41
PING 10.110.156.41 (10.110.156.41) 56(84) bytes of data.
64 bytes from 10.110.156.41: icmp_seq=1 ttl=63 time=26.6 ms
64 bytes from 10.110.156.41: icmp_seq=2 ttl=63 time=3.44 ms
64 bytes from 10.110.156.41: icmp_seq=3 ttl=63 time=40.8 ms
64 bytes from 10.110.156.41: icmp_seq=4 ttl=63 time=30.1 ms
64 bytes from 10.110.156.41: icmp_seq=5 ttl=63 time=9.22 ms
64 bytes from 10.110.156.41: icmp_seq=6 ttl=63 time=12.1 ms
64 bytes from 10.110.156.41: icmp_seq=7 ttl=63 time=7.75 ms
64 bytes from 10.110.156.41: icmp_seq=8 ttl=63 time=5.60 ms
64 bytes from 10.110.156.41: icmp_seq=9 ttl=63 time=18.6 ms
64 bytes from 10.110.156.41: icmp_seq=10 ttl=63 time=32.0 ms
64 bytes from 10.110.156.41: icmp_seq=11 ttl=63 time=8.52 ms
64 bytes from 10.110.156.41: icmp_seq=12 ttl=63 time=18.6 ms
64 bytes from 10.110.156.41: icmp_seq=13 ttl=63 time=19.1 ms
64 bytes from 10.110.156.41: icmp_seq=14 ttl=63 time=9.20 ms
64 bytes from 10.110.156.41: icmp_seq=15 ttl=63 time=8.93 ms
64 bytes from 10.110.156.41: icmp_seq=16 ttl=63 time=26.2 ms
64 bytes from 10.110.156.41: icmp_seq=17 ttl=63 time=8.91 ms
64 bytes from 10.110.156.41: icmp_seq=18 ttl=63 time=7.06 ms
64 bytes from 10.110.156.41: icmp_seq=19 ttl=63 time=18.9 ms
64 bytes from 10.110.156.41: icmp_seq=20 ttl=63 time=9.24 ms
^C
--- 10.110.156.41 ping statistics ---
20 packets transmitted, 20 received, 0% packet loss, time 19028ms
rtt min/avg/max/mdev = 3.436/16.032/40.766/10.088 ms
```

```
Chris-Pi@raspberrypi:~ $ ping 10.110.167.3
PING 10.110.167.3 (10.110.167.3) 56(84) bytes of data.
64 bytes from 10.110.167.3: icmp_seq=1 ttl=63 time=3.31 ms
64 bytes from 10.110.167.3: icmp_seq=2 ttl=63 time=5.29 ms
64 bytes from 10.110.167.3: icmp_seq=3 ttl=63 time=4.52 ms
64 bytes from 10.110.167.3: icmp_seq=4 ttl=63 time=6.25 ms
64 bytes from 10.110.167.3: icmp_seq=5 ttl=63 time=3.81 ms
64 bytes from 10.110.167.3: icmp_seq=6 ttl=63 time=4.05 ms
64 bytes from 10.110.167.3: icmp_seq=7 ttl=63 time=6.95 ms
64 bytes from 10.110.167.3: icmp_seq=8 ttl=63 time=5.48 ms
64 bytes from 10.110.167.3: icmp_seq=9 ttl=63 time=8.33 ms
64 bytes from 10.110.167.3: icmp_seq=10 ttl=63 time=3.58 ms
64 bytes from 10.110.167.3: icmp_seq=11 ttl=63 time=4.33 ms
64 bytes from 10.110.167.3: icmp_seq=12 ttl=63 time=4.99 ms
64 bytes from 10.110.167.3: icmp_seq=13 ttl=63 time=5.57 ms
64 bytes from 10.110.167.3: icmp_seq=14 ttl=63 time=4.39 ms
64 bytes from 10.110.167.3: icmp_seq=15 ttl=63 time=4.84 ms
64 bytes from 10.110.167.3: icmp_seq=16 ttl=63 time=4.20 ms
64 bytes from 10.110.167.3: icmp_seq=17 ttl=63 time=5.91 ms
64 bytes from 10.110.167.3: icmp_seq=18 ttl=63 time=3.69 ms
64 bytes from 10.110.167.3: icmp_seq=19 ttl=63 time=5.28 ms
64 bytes from 10.110.167.3: icmp_seq=20 ttl=63 time=5.26 ms
64 bytes from 10.110.167.3: icmp_seq=21 ttl=63 time=2.43 ms
^C
--- 10.110.167.3 ping statistics ---
21 packets transmitted, 21 received, 0% packet loss, time 20030ms
rtt min/avg/max/mdev = 2.428/4.879/8.325/1.290 ms
```

Sebastian → Chris (AVG = 16.032ms MAX = 40.766ms) Chris → Sebastian (AVG = 4.828ms MAX = 8.325ms)

Has it Been Done before?



Note:

- Uses Raspberry Pi 4 (No crypto engine)
- Does NOT show camera feed

Thank you

**Any
questions?
Ask away!**

